Tema 2 Agentes y Sistemas Multi-Agente

Autor: Ismael Sagredo Olivenza

En el tema anterior

- ¿Qué es la inteligencia Artificial?
- Tipos de inteligencia artificial
- Introducción a Agentes
- Introducción a Sistemas Cognitivos
- Historia de la IA

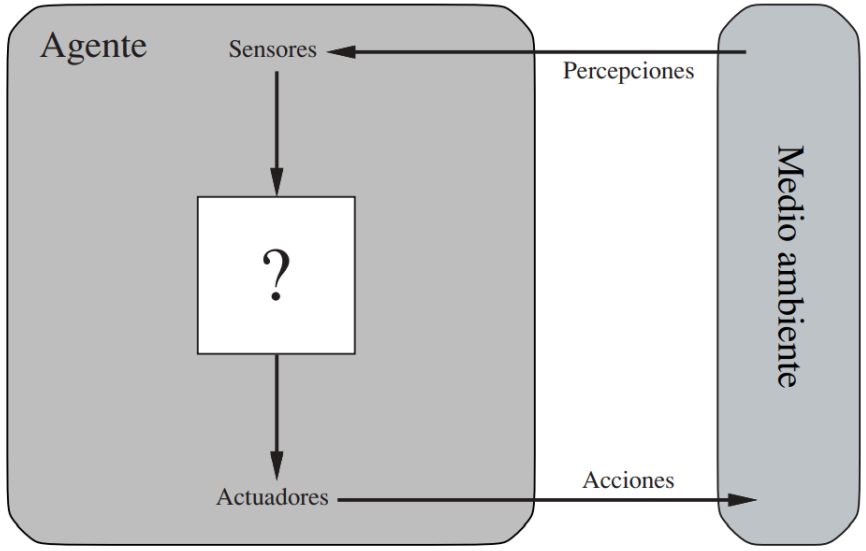
En este tema

- Agentes
- Sistemas Multi-Agente
- Entornos de desarrollo de agentes

Introducción

Ya hemos hablado un poco de agentes, **recordemos**:

Un agente es cualquier cosa capaz de percibir su *entorno* con la ayuda de **sensores** y actuar en ese medio utilizando **actuadores**



Podemos decir que un **agente** es una forma de diseñar software que se caracteriza por :

- Ser un proceso computacional autónomo
- Con capacidad para percibir e interactuar con el entorno
- Que puede comunicarse con otros agentes

Estos sistemas computacionales **suelen habitar en ambientes dinámicos y complejos** y están normalmente diseñados para llevar a cabo **un conjunto finito de tareas**.

El **entorno** puede ser un medio físico (Aspiradora en una habitación) o virtual (NPC en un videojuego).

La **percepción** implica recibir entradas proveniente del exterior del agente. La **secuencia de percepciones** sería el histórico de dichas percepciones.

Una agente tomará una **decisión** en un momento dado en base a la secuencia de percepciones que ha recibido hasta el instante de tomar dicha decisión.

Matemáticamente podemos describir un agente como una **función** que trasforma las **percepciones** en **acciones** del agente.

Formalmente f es una función **sobreyectiva** que mapea percepciones y acciones:

$$f : P \to A$$

Es sobreyectiva porque, todas las acciones deben tener al menos una percepción que las desencadena, pero puede haber casos donde una acción sea desencadenada por más de una percepción.
Donde:

- P es el conjunto de percepciones
- A es el conjunto de acciones

La función se puede describir en forma de tabla (generalmente inviable => un programa que la aproxima y engloba casos comunes en "estados").

Recuerda: un agente racional es aquel que hace lo correcto

- El agente obtiene el mejor resultado
- Necesitamos una medida de **rendimiento** del agente.
  - No hay una medida de rendimiento única para un agente.
  - Medida de rendimiento: mejor fijarse en cómo va a estar el medio (Aspiradora el suelo limpio, en vez de basura recogida)

Racionalidad del agente

La racionalidad en un momento determinado depende de cuatro factores:

- La medida de rendimiento que define el criterio de éxito.
- El conocimiento del medio en el que habita acumulado por el agente.
- Las acciones que el agente puede llevar a cabo.
- La secuencia de percepciones del agente hasta este momento

Irracionalidad: Si por ejemplo un agente hace algo poco lógico.

Racionalidad vs Perfección

Racionalidad no es perfección. **La racionalidad maximiza el rendimiento esperado** mientras que la **perfección maximiza el resultado real**.

Un comportamiento perfecto a veces puede acabar pareciendo irracional (Inhumano)

En ciertos entornos no es problemático, incluso deseable como meta (normalmente es irrealizable porque no se puede tener todo el conocimiento) pero en otros podría hacer que el agente sea rechazado por parecer que "hace trampas".

En ciertos casos, el agente **conoce el entorno en su tutalidad**.

- En estos casos, el agente no necesita aprender.
- Pero en general, el agente necesita almacenar nuevo conocimiento.

Ejemplo: aspirador que debe aprender cómo es tu casa para optimizar sus algoritmos de limpieza. Inicialmente el agente tendra información parcial que necesita completarse con **exploración**.

Se dice que un agente tiene poca **autonomía** si se apoya en el conocimiento inicial que le proporciona el diseñador más que en su percepción. Agente racional => Debe ser autónomo.

Un agente racional debería tener un conocimiento inicial y una forma de adquirir nuevo conocimiento. El conocimiento inicial le sirve para poder sobrevivir hasta adquirir el conocimiento que necesita.

Ejemplo:

- Aspiradora: inicialmente se moverá aleatoriamente o siguiendo alguna rutina prefijada. Cuando ya construya el plano de la casa podrá realizar mejores planes de limpieza y desechará las rutinas iniciales.
- Robot marciano: debe poder sortear accidentes geográficos y tener rutinas de exploración del medio. Después de aprender => Puede explorar mejor y detectar donde debe realizar los experimentos.

Entorno

El entorno o medio ambiente es fundamental para un agente. En el diseño de un agente, el primer paso debe ser siempre especificar el entorno de trabajo de la forma más completa posible.

Formalización del entorno:

REAS

(**R**endimiento, **E**ntorno, **A**ctuadores, **S**ensores)

REAS

- **Rendimiento**: definir una medida de rendimiento.
  - Clave para evaluar el agente y que éste haga lo que queremos.
  - Debe beneficiar o penalizar las acciones para que le sea útil
- **Entorno**: Descripción y formalización del entorno.
- **Acciones**: Descripción de qué cosas puede modificar el agente en el entorno y cómo afecta a este.
- **Sensores**: Descripción de cómo percibe el agente el entorno

REAS

- Modelemos con REAS un vehículo autónomo
- Modelamos con REAS un NPC de Metal Gear Solid

Propiedades del entorno

- **Totalmente vs parcialmente observable**: Si sensores proporcionan toda la información relevante del medio. (Ajedrez vs Poker)
- **Determinista vs estocástico**: Si el siguiente estado del medio está totalmente determinada por el estado actual y la acción del agente (Ajedrez vs Poker)
- **Episódico vs secuencial**: Secuencial si una decisión del agente puede afectar a decisiones futuras. (Detección de piezas vs Ajedrez)
- **Estático vs dinámico**: Si puede haber cambios en el entorno

- **Discreto vs Continuo**: Discreto => Nº finito de estados (Ajedrez vs Conducción)
- **Individual vs multiagente**: Si en el sistema opera un único agente o un conjunto. (Aspiradora vs Fútbol)
- **Competitivo vs colaborativo**: Si los agentes compiten o colaboran. (Poker vs hormigas) (Puede haber ambos, por ejemplo la brisca)

Ejercicio: ¿Cual es el caso más complejo? Pon un ejemplo.

¿De qué tipo son estos entorno?

- Damas:
- Age of Empire:
- Aspiradora:
- Robot sexador de pollos:
- El parchís:
- Tetris:
- Exploración de marte usando un robot:
- Partido de fútbol en el FIFA.

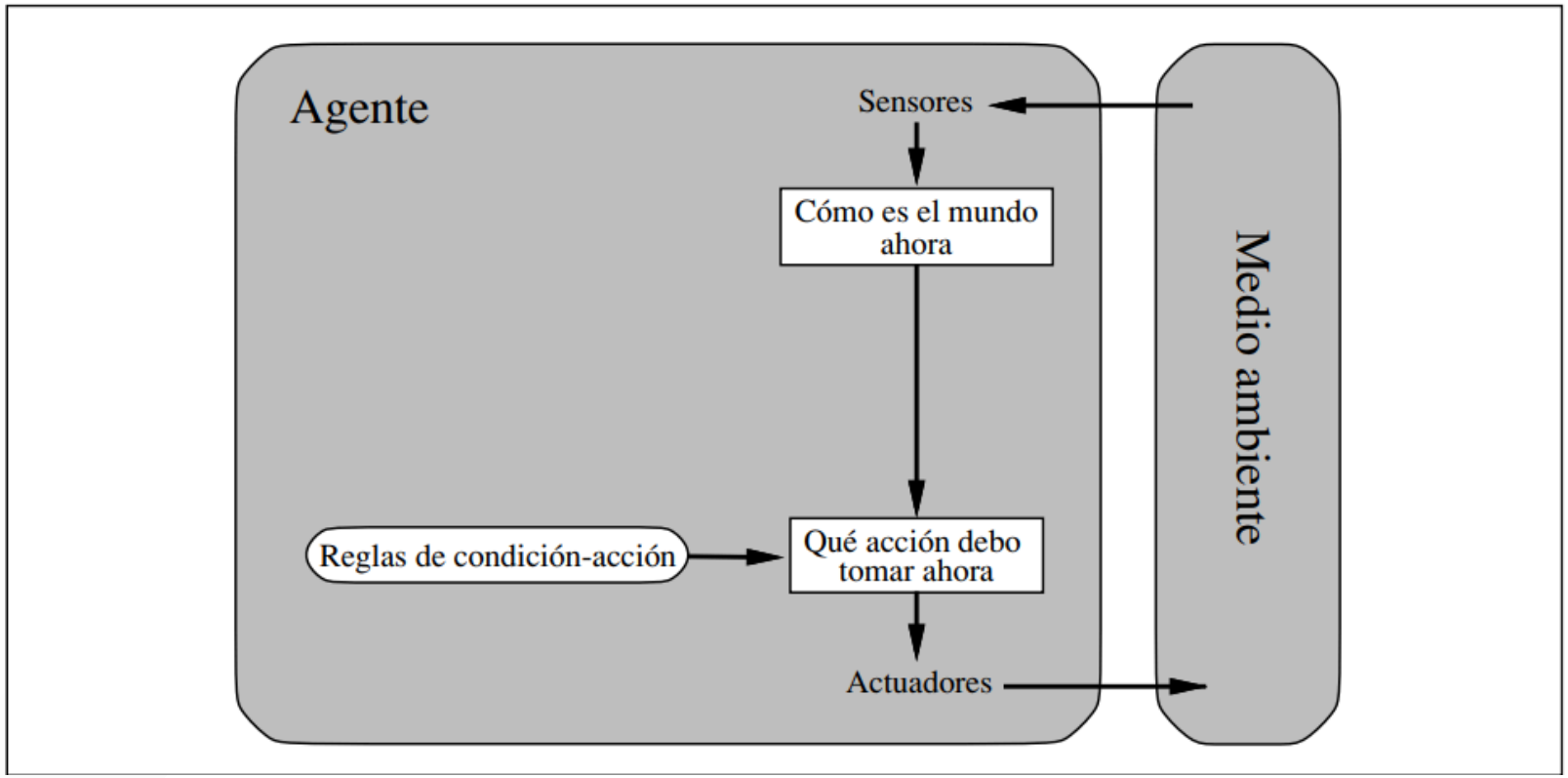
Arquitectura de agentes

Agente esta formado por:

- Arquitectura del agente: Sensores, actuadores y si es físico, hardware para procesar el control.
- Control del agente: El sistema que controla el agente, normalmente software pero podría ser hardware (FPGA, circuito lógico integrado)

Tipos de agentes según su control

Agente reactivo Simple



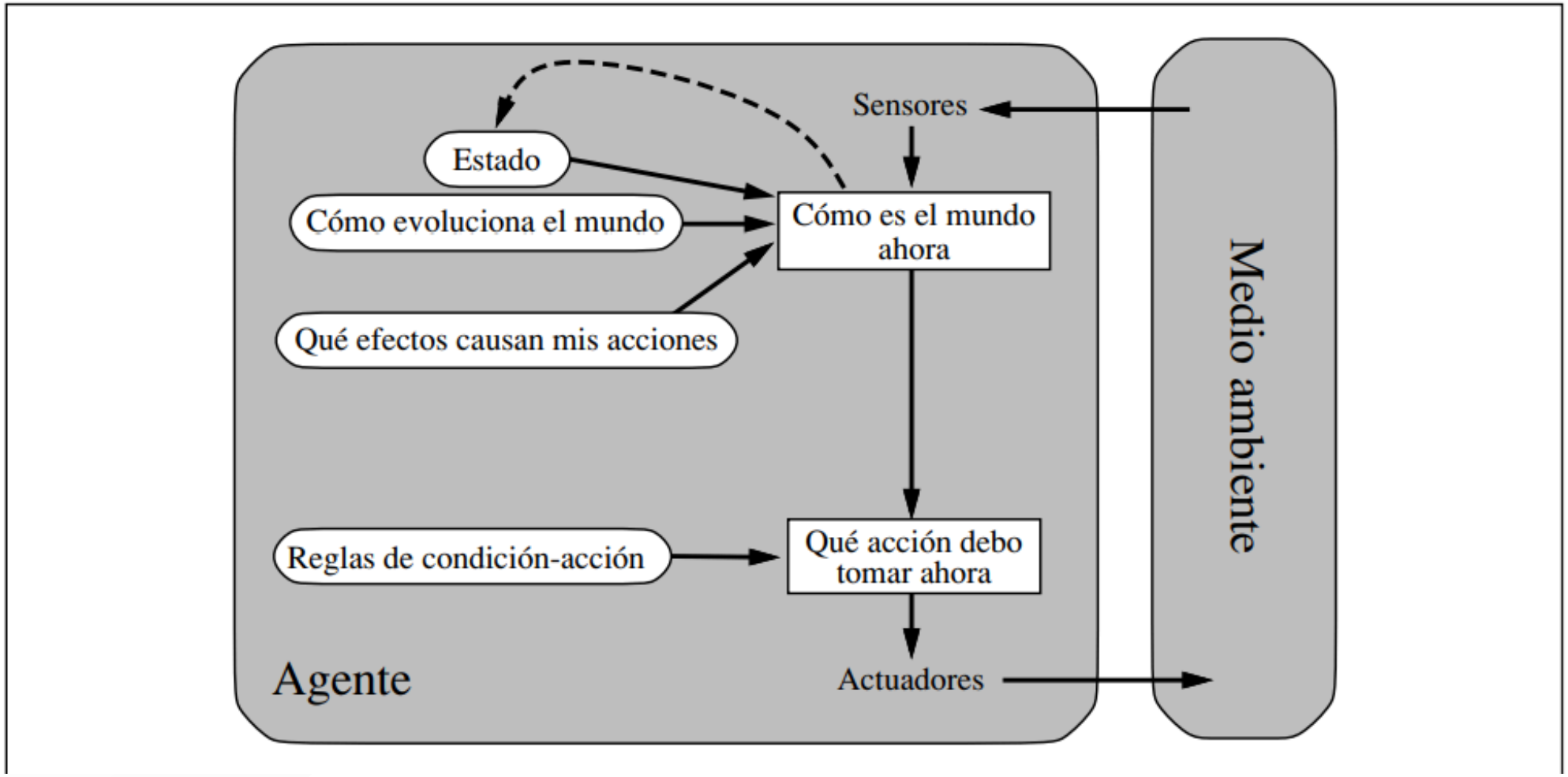
Agente reactivo Simple

- Formalismos: Tabla, if-else, interprete de reglas básicas.

```
función AGENTE-ASPIRADORA-REACTIVO([localización, estado]) devuelve una acción
  si estado  Sucio entonces devolver Aspirar
  de otra forma, si localización  A entonces devolver Derecha
  de otra forma, si localización  B entonces devolver Izquierda
```

- Decisiones sólo en base a la percepción, sin ningún histórico.
- Humanos tienen este tipo de respuestas reactivas => respuestas rápidas a eventos repentinos.
- Simples pero muy limitados, propenso a bucles infinitos (salida usando random)

Agente reactivo basado en modelo



Agente reactivo basado en modelo

Formalismos: Máquina de estados finito, a pila, árboles de comportamiento

```
función AGENTE-REACTIVO-CON-ESTADO(percepción) devuelve una acción
  estático: estado, una descripción actual del estado del mundo
  reglas, un conjunto de reglas condición-acción
  acción, la acción más reciente, inicialmente ninguna

  estado <= ACTUALIZAR-ESTADO(estado, acción, percepción)
  regla <= REGLA-COINCIDENCIA(estado, reglas)
  acción <= REGLA-ACCIÓN[regla]
  devolver acción
```

Agente reactivo basado en modelo

- Mantienen estado interno que depende de la historia de percepción.
- El estado debe actualizarse de alguna forma.
- Guardar la información parcial que no percibes o que pueda ser obsoleta
- La meta en estos agentes es intrínseca y no cambia (Voy a limpiar)

Ejercicio:

Maquina de estados que modela el comportamiento de un Goomba.

Deambula por el mundo de forma psudo-aleatoria. Si encuentra una pared o se sale de los limites establecidos, gira 180º y se muve durante un rato en dirección contraria al obstaculo. Despues vuelve a deambular. Si detecta al jugador se sorprende, una vez terminada la animación de sorpresa le persigue. Si el jugador salta sobre él muere.

Ejercicio:

Maquina de estados que simula el comportamiento de un Agente en un fps.

El agente busca por el mapa a un adversario para dispararle. Cuando lo ve dispara al adversario con su arma cargada. Si el arma no tiene el alcance suficiente cambia a un arma más apropiada para el disparo. Si le queda poca vida intenta buscar un botiquín cercano abandonando cualquier acción que estuviera haciendo en ese momento (excepto cambiar de arma que es bloqueante). Si no hubiese ningún botiquín a la vista, el agente va hacia el adversario para atacarle cuerpo a cuerpo.

¿Cómo se implementa una máquina de estados?

- Switch case
- Con POO
- Dirigida por datos

Variantes:

- Maquina de estados jerárquica
- Maquinas de estado probabilísticas

Behavior Tree

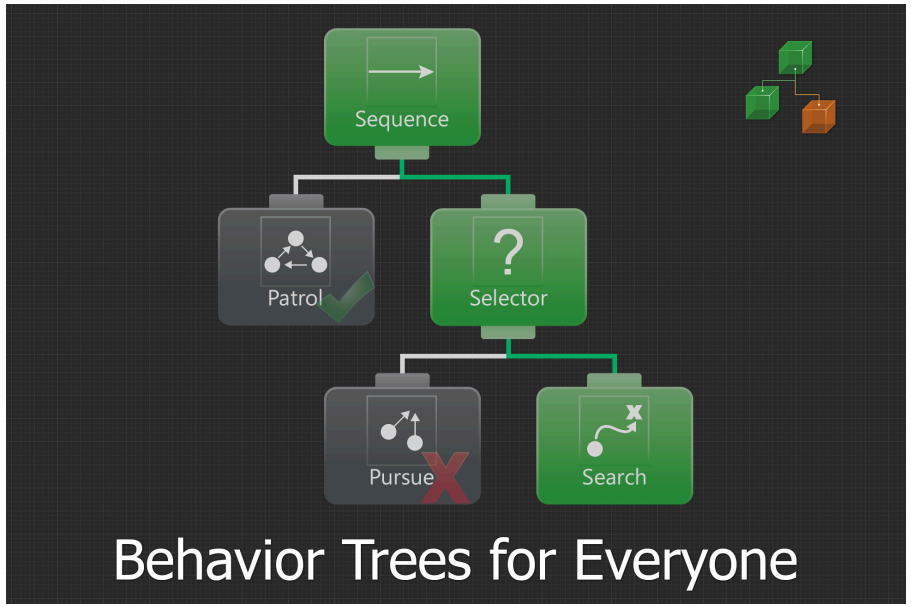
Ejecuta un árbol de comportamiento desde la raíz hasta las hojas de arriba a abajo ude izquierda a derecha. Los nodos no hoja son nodos de decisión, los nodos hoja o terminales son nodos de acción.
Todosl os nodos peude terminar en dos estados: **success o failure**

Los nodos de decisión pueden ser de diferentes tipos:

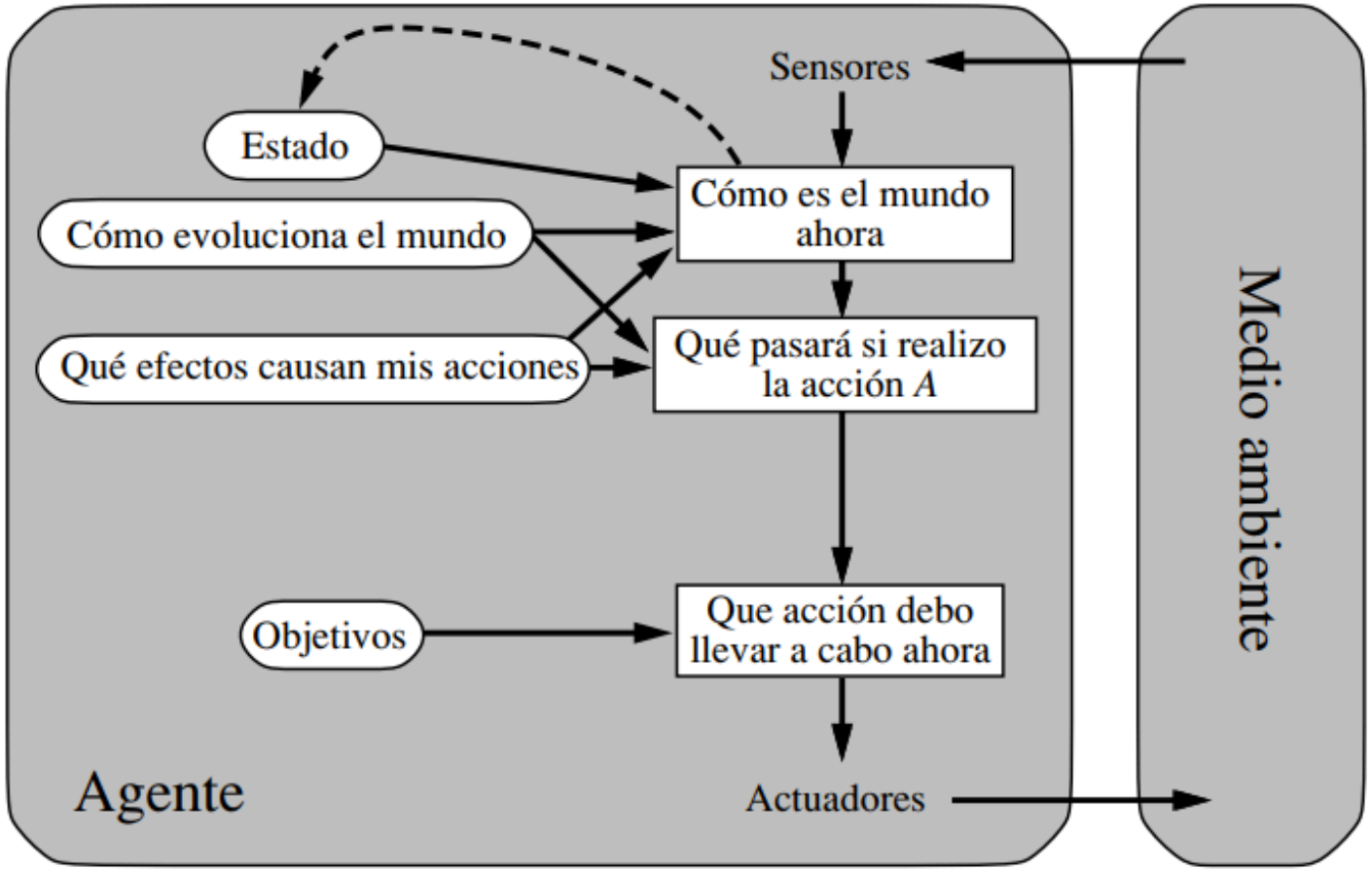
- secuencia: ejecuta todos los nodos uno tras otro mientras sus estados de terminación sean success.
- selector: ejecuta todos los nodos uno tras otro mientras sus estados sean failure. Termina ocn el primer hijo success.

- Parallel: ejecuta n nodos sin tener en cuenta su estad ode terminación
- ETC: Se pueden definir nodos como queramos.

Los nodos temrinales puede ser acciones o condiciones que si se cumplen devuelven success y si no failure.



Agentes basados en objetivos

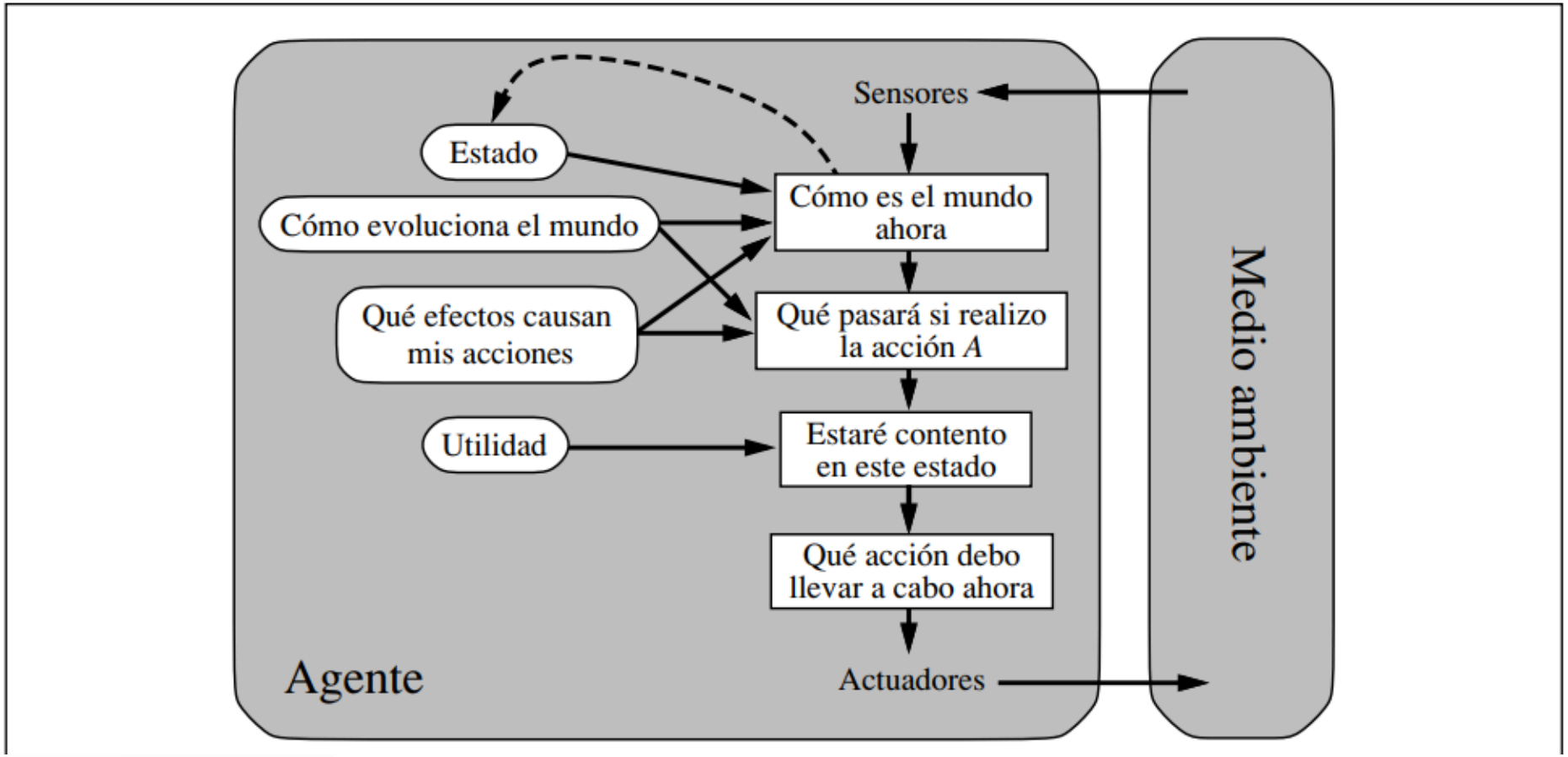


Agentes basados en objetivos

Formalismos: Búsqueda, Planificación etc.

- **Las metas pueden cambiar** por lo que se requiere replanificar.
- Al no existir una meta implícita, una misma decisión será o no acertada en función de la meta.

Agentes basados en utilidad

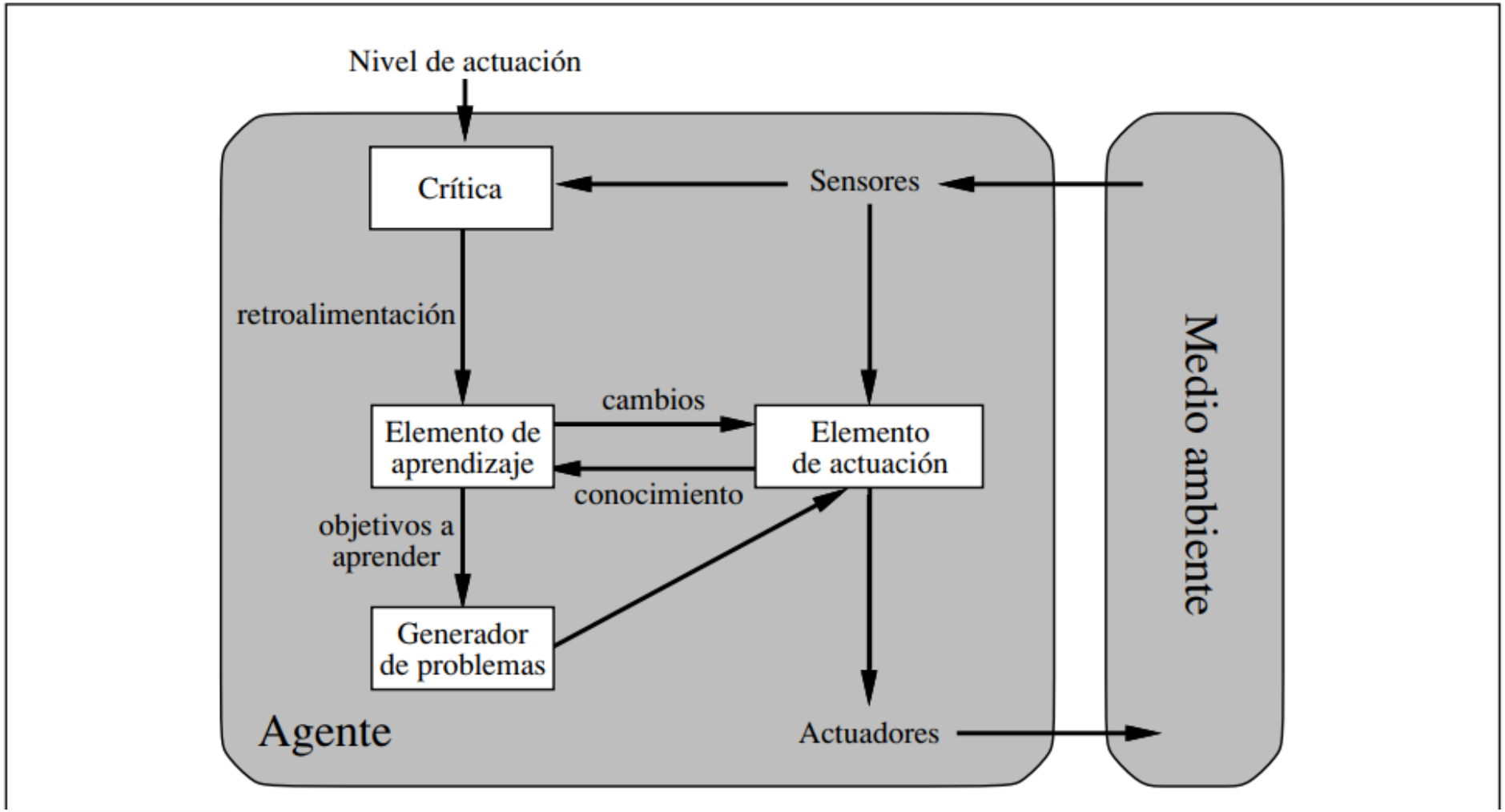


Agentes basados en utilidad

Formalismos: Función de utilidad

- Permite elegir mejor entre diferentes opciones para llegar a un objetivo.
- Permite decidir entre objetivos contradictorios (Llegar rápido o llegar de forma segura)
- Si no tenemos certezas, la función de utilidad permite establecer probabilidades.

Agentes que aprenden



Agentes que aprenden

Formalismos: Aprendizaje por refuerzo, supervisado y no supervisado

- Permite al agente moverse por entornos desconocidos.
- Permite a un mismo agente resolver diferentes problemas
- Elemento de actuación es cualquiera de los agentes anteriores
- La critica observa si lo que se está haciendo es correcto
- Elemento de aprendizaje formula nuevas reglas y actualiza la actuación
- Generador de problemas debe investigar como mejorar

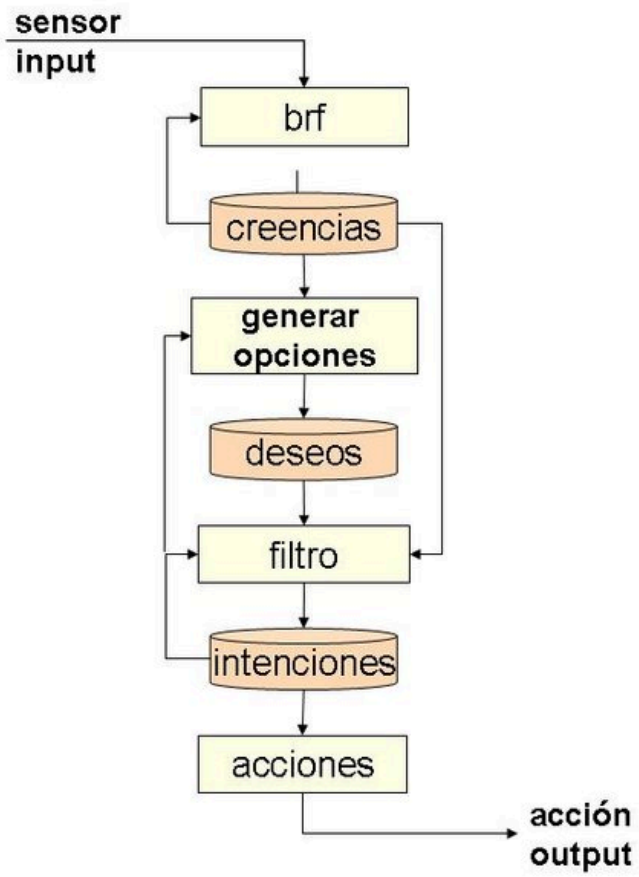
Agentes deliberativos basados en objetivos

Hay varias formas de implementar un agente basado en objetivos:

- Usando reglas de inferencia (Lo veremos más adelante)
- Usando búsqueda y planificación (El siguiente tema :))
- Sistema intencionales usando la arquitectura **BDI** (Belief, Desire, Intention) (Bratman, 1987) (Georgeff, et al., 1998)
  - **Creencias (Beliefs)**: Lo que el agente conoce.
  - **Deseos (Desires)**: Son las metas
  - **Intenciones (Intentions)**: La deliberación del agente actual, plan

Arquitectura BDI

- BRF: función de revisión de creencias
- Función generadora de opciones
- Un filtrado de opciones para quedarnos con la intención actual



Fuente: Ponce, Julio et al. (2014). Inteligencia Artificial.

Agentes vs Objetos

¿Por que usar agentes y no objetos?

- Los objetos son pasivos ya que no actúa hasta recibir un mensaje.
- Los objetos no toman decisiones
- Los objetos obedecen órdenes
- POO es una solución adecuada par entornos conocidos y bien definidos.

Se puede utilizar un paradigma de orientación a objetos para implementar uno de agentes

Sistemas multi-agentes

Sistema Multi-agente (MAS) es un conjunto de agentes autónomos, normalmente heterogéneos que trabajan en resolver un problema entre todos ellos.

- Inteligencia artificial distribuida. (DAI)
- Necesitan comunicarse para resolver los problemas
- Pueden tener información parcial y redundancia de información

Aunque lo más normal es que el agente sea autónomo y tome sus propias decisiones, se puede tener una coordinación centralizada

Ventajas de los SMA

- Normalmente aumenta la potencia del sistema
- Robustez y tolerancia a fallos
- Sinergias entre agentes (Comportamiento emergente)
- No es necesario que el agente conozca toda la información (Perfiles de seguridad)
- Se adapta mejor a entornos dinámicos

Interacción entre los agentes

Los agentes deben interactuar entre si en base a tres parámetros:

- Objetivos de los agentes que interactúan: compatibles / incompatibles
- Recursos disponibles para realizar las tareas: suficientes / insuficientes
- Habilidades del agente para realizar la tarea: suficientes / insuficientes

Tipos de colaboración

Tipo	Objetivos	Recursos	Habilidad
Independencia	Compat	Suf	Suf
Colaboración simple	Compat	Suf	Insuf
Obstrucción	Compat	Insuf	Suf
Colaboración coordinada	Compat	Insuf	Insuf
Competición individual	Incompat	Suf	Suf
Competición por equipos	Incompat	Suf	Insuf
Conflicto individual	Incompat	Insuf	Suf
Conflicto colectivo	Incompat	Insuf	Insuf

Ejemplos

- Los jugadores del Mus
- Brokers online
- Drones fireworks
- Coches autónomos
- Agentes de compra online

Organización de los agentes

Es fundamental en las sociedades de agentes. Se debe definir el número y tipo de agentes, el rol de cada agente y sus relaciones.

- Organización definida por el programador
- Organización emergentes como resultado de la interactividad
- Organización jerárquica
- Organización igualitaria

Las relaciones pueden ser **estáticas** o **dinámicas**.

Tipos de relaciones entre agentes

- Conocimiento: A conoce a B y puede comunicarse con B
- Comunicación: Existe un canal de comunicación entre A y B
- Subordinación: B realiza una tarea que A le pide:
  - Si no se puede negar estática (Maestro / Esclavo)
  - Si se puede negar o negociar dinámica
- Operativa: A depende de que B realice antes otra tarea
- De información: A cree a B
- Conflicto: Varios agentes necesitan mismos recursos
- Competición: Agentes con objetivos incompatibles

Cooperación

Varias formas de cooperar:

- Cooperación intencional: los agentes son conscientes de que deben colaborar
- Cooperación desde el punto de vista del observador (los agentes no son conscientes)

La cooperación es útil para mejorar el rendimiento del grupo y resolver conflictos.

Tipos de cooperación

- Agrupamiento de agentes: Biología
- Compartir información
- Especialización de tareas
- Compartir recursos
- Resolución de conflictos mediante arbitraje

La comunicación

Es la base de la colaboración y la coordinación. Los agentes se comunican para solicitar tareas, recursos, proponer información, compartir creencias o coordinarse para realizar una tarea.

- **Señal**: modificación del entorno. No tiene destinatario fijo
- **Mensaje**: el agente envía el mensaje a otro agente. Tiene un destinatario
  - Síncronos: Se espera a que el receptor conteste inmediatamente
  - Asíncrono: No se espera la recepción del mensaje y se retarda en el tiempo

Tipos de colaboración

- Reparto de tareas:
  - Distribuida: cada agente gestiona los recursos y servicios
  - Centralizada: Un agente coordinador se encarga del reparto
  - Emergente: El reparto surge de las interacciones.
- Contrato: el proveedor se compromete a realizar una tarea y el cliente a no encargársela a otro. Los contratos pasan por fases (asignado, cerrado, hecho, roto)
  - **Smart contracts**: contratos que realizan agentes software que tiene validez. Están muy extendidos últimamente y se apoyan en **blockchain**.

- Red de contratos:
  - Se publica una tarea
  - Contratistas envían ofertas (red de conocidos, registro, memoria)
  - Cliente evalúa las ofertas y se queda con la que más le interesa
  - Se formaliza el contrato
- Cosas a especificar en un contrato:
  - Calidad mínima requerida
  - Como se evaluará
  - Fecha límite de realización

Se necesita un protocolo común de comunicación

Para que se entiendan correctamente dos máquinas, ambas deben hablar el mismo lenguaje.

Para ello se crearon diferentes estandares. La FIPA (1996-2005) creo el denominado Standar FIPA con el objetivo de construir agentes interoperables.
El estándar solo fija el comportamiento externo de los agentes: **ACL framework for agents**
Actualmente está integrado en la IEEE Computer Society

Entornos de desarrollo

- AgentPy: https://agentpy.readthedocs.io/en/latest/index.html
- PADE: https://pypi.org/project/pade/
- JADE: https://jade.tilab.com/
- GAMA: https://gama-platform.org/

¿Dónde se usan los agentes y sistemas multiagente?

- Drones
- Bots de búsqueda por la web
- Agentes de mediación (Brokers)
- Agentes asistentes personales (Planificadores de agendas, salas de emergencia)
- Agentes de interfaz
- Resolución de problemas en entornos dinámicos

Bibliografía

- Russell, S. J., & Norvig, P. (2004). Inteligencia Artificial: un enfoque moderno (No. 04; Q335, R8y 2004.).
- Jacques Ferber. Multi-agent Systems- An introduction to distributed artificial intelligence. 1999 Addison-Wesley
- Ali Dorri, Multi-Agent Systems: A Survey (1998) IEEE Access https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8352646